

Lambda, Map, Iterators and Generators in Python

Contents

1	Introduction	3
2	Lambda Function	3
3	Normal Function	3
4	Lambda Version	3
5	Why Lambda?	3
6	Higher Order Functions	4
7	map() Function	4
8	Example: Converting Strings to Integers	4
9	Lazy Evaluation	4
10	Using next()	4
11	Custom Function in map	5
12	Using map with Custom Function	5
13	Using Lambda in map	5
14	filter() Function	5
15	What is filter()?	5
16	Syntax	5
17	Simple Example	6
18	Using Normal Function	6
19	Using Lambda Function	6
20	Difference between map() and filter()	6

21 Example Comparison	6
21.1 map() Example	6
21.2 filter() Example	6
22 sorted() Function	7
23 Sorting Dictionary	7
24 Sorting by Values	7
25 How Sorting Works	7
26 Iterator Concept	7
27 Creating Iterator	7
28 Using next()	7
29 StopIteration	8
30 Handling Iterator Safely	8
31 What is Generator?	8
32 Normal Function vs Generator	8
33 Yield Keyword	8
34 Simple Generator	9
35 Using Generator	9
36 Infinite Generator Example	9
37 Why Generators?	9
38 Custom Range Generator	9
39 Difference: List vs Generator	10
40 Real-world Use	10
41 Final Summary	10

1 Introduction

Till now, we learned:

- Functions
- Loops
- Comprehensions
- File handling
- Exception handling

Now we move to more advanced concepts:

Functional Programming Tools

These help us write:

- shorter code
- faster processing
- memory-efficient programs

2 Lambda Function

Lambda function is:

Anonymous (no name) function

3 Normal Function

```
def cube(x):  
    return x**3
```

4 Lambda Version

```
cube = lambda x: x**3
```

5 Why Lambda?

Used when:

- function is small
- function is temporary
- function is passed as argument

6 Higher Order Functions

Functions that take:

- other functions as input

Examples:

- `map()`
- `filter()`
- `sorted()`

7 `map()` Function

```
map(function, iterable)
```

It applies function to every item.

8 Example: Converting Strings to Integers

```
marks = ['45', '34', '78', '86']  
  
int_marks = list(map(int, marks))
```

9 Lazy Evaluation

```
int_marks = map(int, marks)
```

This does NOT compute immediately.

It creates:

Iterator

Values are generated only when needed.

10 Using `next()`

```
next(int_marks)
```

Each call gives next value.

This is called:

Lazy evaluation

11 Custom Function in map

```
def pass_or_fail(mark):  
    if mark >= 30:  
        return 'P'  
    else:  
        return 'F'
```

12 Using map with Custom Function

```
marks_to_pass_fail = map(pass_or_fail, int_marks)
```

13 Using Lambda in map

```
marks_to_pass_fail = map(  
    lambda mark: 'P' if mark >= 30 else 'F',  
    int_marks  
)
```

14 filter() Function

After learning `map()`, we now look at another very important higher-order function:

`filter()`

15 What is filter()?

`filter()` is used to:

select only those items that satisfy a condition

Unlike `map()`, which transforms every item, `filter()` removes unwanted items.

16 Syntax

```
filter(function, iterable)
```

The function must return:

- True → keep the item
- False → remove the item

17 Simple Example

```
nums = [1, 2, 3, 4, 5, 6]
```

We want only even numbers.

18 Using Normal Function

```
def is_even(num):  
    return num % 2 == 0  
  
even_nums = list(filter(is_even, nums))
```

19 Using Lambda Function

```
even_nums = list(filter(lambda x: x % 2 == 0, nums))
```

20 Difference between map() and filter()

- `map()` → transforms every element
- `filter()` → selects elements based on condition

21 Example Comparison

21.1 map() Example

```
nums = [1, 2, 3, 4]  
  
squares = list(map(lambda x: x**2, nums))
```

Output:

```
[1, 4, 9, 16]
```

21.2 filter() Example

```
nums = [1, 2, 3, 4]  
  
even = list(filter(lambda x: x % 2 == 0, nums))
```

Output:

```
[2, 4]
```

22 sorted() Function

```
sorted(iterable, reverse=True)
```

Returns new sorted list.

23 Sorting Dictionary

```
stu_marks = {  
    "ram": 89,  
    "hari": 78,  
    "seeta": 99  
}
```

24 Sorting by Values

```
sorted(stu_marks.items(), key=lambda item: item[1])
```

25 How Sorting Works

Each item looks like:

```
("ram", 89)
```

Lambda selects:

```
item[1]
```

which is marks.

26 Iterator Concept

Iterator means:

Object that gives values one by one

27 Creating Iterator

```
mi = iter(marks)
```

28 Using next()

```
next(mi)
```

Returns next element.

29 StopIteration

When data ends:

StopIteration error occurs

30 Handling Iterator Safely

```
while True:
    try:
        value = next(mi)

    except StopIteration:
        break

    else:
        print(value)
```

31 What is Generator?

Generator is:

Function that produces values one by one using yield

32 Normal Function vs Generator

Normal function:

- returns once

Generator:

- yields multiple times

33 Yield Keyword

```
yield 1
yield 2
yield 3
```

Each yield:

- pauses function
- saves state
- resumes later

34 Simple Generator

```
def num_gen():  
    yield 1  
    yield 2  
    yield 3
```

35 Using Generator

```
nums = num_gen()  
  
next(nums)
```

36 Infinite Generator Example

```
def even_num_generator():  
    start = 2  
  
    while True:  
        yield start  
        start += 2
```

37 Why Generators?

They are used because:

- memory efficient
- good for large data
- used in streaming systems

38 Custom Range Generator

```
def range(stop):  
    start = 0  
  
    while start < stop:  
        yield start  
        start += 1
```

39 Difference: List vs Generator

List:

- stores all values in memory

Generator:

- generates values on demand

40 Real-world Use

Generators are used in:

- large datasets
- file reading
- APIs
- streaming services

41 Final Summary

Today you learned:

- Lambda functions
- `map()` and filter concepts
- sorted with key
- iterators
- `next()` and `StopIteration`
- generators
- yield keyword

These concepts are heavily used in:

Modern Python development and frameworks